

RAP Cloud 課程 4：Action (自訂操作)

Lecture

這一課要證明的事情

rap02 ~ rap07 (舊 On-Premise 課程) 教的 Determination / Validation，因為系統版本落後，Handler Method 被迫用官方標成 obsolete 的 `FOR DETERMINATION / FOR VALIDATION` 語法 (見舊課程 rap05 / rap06、這門課 rc03)。Action 剛好相反——舊課程 rap07 當年就已經查證並實測確認：Action 的 Handler Method 完全不需要 obsolete 轉換，官方現行語法 `FOR MODIFY ... FOR ACTION ...` 直接就能編譯。

這一課的任務是：在既有的 `ZI_RC01_TASK` (rc02 / rc03 用過的同一個 Managed BDEF) 上加一個 `markDone` Action，逐字比對這個 Cloud 環境的語法跟舊課程 rap07 的語法，驗證「Action 沒有新舊版本落差」這個結論在完全不同的系統版本上依然成立，並把兩組語法 (BDEF 宣告、Handler Method 宣告) 的每一個組成部分講清楚。

版本比對結論先講：完全相同，一字不差

	舊課程 rap07 (On-Premise 1909)	這一課 rc04 (Cloud，BTP ABAP Environment 最新版)
BDEF Action 宣告	<code>action markDone result [1] \$self;</code>	<code>action markDone result [1] \$self;</code>
Handler Method 宣告	<code>METHODS markDone FOR MODIFY
IMPORTING keys FOR ACTION Task~markDone RESULT result.</code>	<code>METHODS markDone FOR MODIFY
IMPORTING keys FOR ACTION Task~markDone RESULT result.</code>
方法本體用的 EML	<code>MODIFY ENTITIES ... IN LOCAL MODE / READ ENTITIES ... IN LOCAL MODE</code>	相同
EML EXECUTE 呼叫語法	<code>EXECUTE markDone FROM VALUE #((%key-... = ...))</code>	相同

這不是巧合，是有原因的一致——原因見下一節。這也是這門課到目前為止第一次「兩個環境的語法完全沒有任何差異」，跟 rc02 (`field(readonly:update)` 這個 Cloud 才支援的語法)、rc03 (Determination / Validation 用官方現行語法而非 obsolete) 都不一樣：rc02 / rc03 講的是「這個 Cloud 環境比舊系統新」，這一課講的是「有些語法元素從一開始就沒有新舊版本可言」。

為什麼 Action 沒有新舊語法落差——回顧 rc03 已經建立的判斷原則

rc03 已經確認一個關鍵事實：Determination / Validation 有自己專屬的 Handler 方法類別 (`FOR DETERMINE ON SAVE / FOR VALIDATE ON SAVE`)，這系統的剖析器只認得到這兩個類別較舊的版本 (`FOR DETERMINATION / FOR VALIDATION`)；但 `create/update/delete` 這個 `FOR MODIFY` 類別本身，從 rc02 開始就一路確認官方現行語法可以直接用，沒有 obsolete 問題。

Action 沒有自己專屬的 **Handler** 類別——它是掛在 **FOR MODIFY** 底下的，用 **FOR ACTION <alias>~<action 名稱>** 子句表明「這個方法對應的是哪一個 Action」，本質上跟宣告一個 **create/update/delete** 方法是同一套機制、同一個語法家族。既然 **FOR MODIFY** 這個類別本身沒有新舊版本問題，掛在它底下的 Action 自然也不會有。

這也解釋了為什麼「新舊語法落差」不是全面性的——不能簡單說「這個系統支援新語法」或「那個系統只認舊語法」，落差是精確發生在**特定 Handler 類別**這個層級。遇到新的 RAP 語言元素，正確做法永遠是：先查這個元素屬於哪個 Handler 類別，再判斷這個類別過去是否踩過新舊語法的坑，不要籠統地套用「這個系統版本新，所以什麼都能用」的假設。

BDEF 語法逐項詳解：**action markDone result [1] \$self;**

查證官方文件 **ABENBDL_ACTION_NONFACTORY** 的完整語法定義：

```
[internal] [static] [repeatable] action
  [ ( [features: {instance|global}] [precheck]
[authorization:none|update|global|instance] [lock:none] ) ]
  ActionName [external 'ExternalName']
  [InputParameter]
  [OutputParameter]
```

這一課實際用的是這個語法的最簡形式，逐項拆解：

組成部分	這一課的寫法	意義
action	action	關鍵字，宣告這是一個自訂操作（跟 create/update/delete 這些標準操作不同，Action 代表一個有業務含義的動作）
（修飾詞，這一課全部省略）	無	internal （限定只能從 BO 內部呼叫，外部不可見）、 static （宣告成不綁定特定實例的靜態操作，例如「批次核准」）、 repeatable （允許同一個 EML/OData 請求裡對同一實例重複執行這個 Action）——這一課的 markDone 是最單純的 instance-bound、一次性操作，都不需要
（...）括號內的選項（這一課省略）	無	例如 authorization:none （排除這個 Action 的權限檢查，但這一課的 BDEF 本來就整個宣告 authorization master(none) ，不需要再對單一 Action 額外宣告）、 precheck （阻擋不該進入緩衝區的變更）、 features:instance/global （動態控制這個 Action 在什麼條件下才能被呼叫，呼應 rc03 沒深入的 field(features:instance) 同一套機制）
markDone	markDone	ActionName ——這個 Action 的名稱，習慣用駝峰式，語意上代表「標記完成」這個業務動作
InputParameter （這一課省略）	無	如果 Action 需要外部傳入資料（例如「核准並附上核准意見」），會在這裡宣告一個帶型別的輸入參數； markDone 不需要任何輸入，所以省略

組成部分	這一課的寫法	意義
<code>result [1] \$self</code>	<code>result [1] \$self</code>	<code>OutputParameter</code> 的一種寫法—— <code>result</code> 是關鍵字，宣告這個 Action 執行完要回傳結果； <code>[1]</code> 是回傳基數 (Cardinality)，代表固定回傳 1 筆； <code>\$self</code> 是型別，代表「回傳型別就是這個實體自己」(也可以指定成另一個型別，例如回傳一個統計摘要，這一課不需要)
<code>;</code>	<code>;</code>	BDL 陳述式結尾

Handler Method 語法逐項詳解：`FOR MODIFY ... FOR ACTION ... RESULT result`

```
METHODS markDone FOR MODIFY
IMPORTING keys FOR ACTION Task~markDone RESULT result.
```

組成部分	意義
<code>METHODS markDone</code>	一般 ABAP 方法宣告，方法名稱習慣跟 BDEF 宣告的 Action 名稱一致 (不強制，但這是慣例，方便對照)
<code>FOR MODIFY</code>	宣告這是一個 RAP Handler Method，屬於 <code>FOR MODIFY</code> 這個類別——跟 <code>create/update/delete</code> (<code>rc02</code> 已經教過) 完全共用同一個類別，這是 Action 沒有新舊語法問題的根本原因
<code>IMPORTING keys</code>	呼叫端透過 EML <code>EXECUTE</code> 傳進來、要對哪些實例執行這個 Action 的識別資訊， <code>keys</code> 是慣例命名 (不是保留字，但幾乎所有官方範例都這樣叫)
<code>FOR ACTION Task~markDone</code>	指明這個方法對應到 BDEF 裡 <code>alias Task</code> 底下宣告的哪一個 Action (<code>markDone</code>)—— <code>~</code> 前面是實體別名，後面是 Action 名稱，這個組合式參照方式在 <code>rc03</code> 的 <code>FOR ... Task~setCreationInfo</code> 已經看過同樣的規律
<code>RESULT result</code>	因為 BDEF 宣告了 <code>result [1] \$self</code> ，這裡對應要宣告一個 <code>RESULT</code> 參數接收輸出；如果 BDEF 的 Action 沒宣告 <code>result</code> 子句，這裡也要省略——兩邊的簽章要對得上

方法本體：把 `IN LOCAL MODE` 這條 `rc03` 學到的規則再用一次

```
METHOD markDone.
MODIFY ENTITIES OF zi_rc01_task IN LOCAL MODE
ENTITY Task
UPDATE FIELDS ( status )
WITH VALUE #( FOR key IN keys
               ( %key = key-%key
                 status = 'D' ) )
FAILED DATA(lt_failed)
REPORTED DATA(lt_reported).
```

```

failed    = CORRESPONDING #( DEEP lt_failed ).
reported  = CORRESPONDING #( DEEP lt_reported ).

READ ENTITIES OF zi_rc01_task IN LOCAL MODE
  ENTITY Task
  FIELDS ( task_id description status priority due_date created_at created_by )
  WITH CORRESPONDING #( keys )
  RESULT DATA(lt_task).

result = VALUE #( FOR ls_task IN lt_task
                  ( %key = ls_task-%key %param = ls_task ) ).
ENDMETHOD.

```

- 邏輯分三步：① 用 **MODIFY ENTITIES ... IN LOCAL MODE** 把 **status** 改成 'D' (合法值——重用 rc03 已經建立的 **validateStatus** 邏輯，'D' 不會被擋下來，示範 Action 跟 Validation 在同一個 RAP BO 裡自然配合，不會互相打架) ② 把這次內部呼叫自己產生的 **failed/reported** 轉存到 Handler Method 自己要回傳的 **failed/reported** (用 **CORRESPONDING #(DEEP ...)** 深層複製，因為兩邊的衍生型別結構一致但不是同一個型別) ③ **READ ENTITIES** 把改完的完整資料讀回來，組成 **result**
- 內部呼叫自己的 **Entity**，一樣要 **IN LOCAL MODE**——這是 rc03 已經建立的規則 (Eclipse 編輯器會主動提示)，Action 的 Handler Method 內部呼叫跟 Determination / Validation 是同一個道理，不因為換了機制就不適用
- **RESULT** 衍生型別每一列有 **%key** (識別是哪一筆實例) 跟 **%param** (實際要回傳的資料內容，型別是 BDEF 宣告的 **\$self**，也就是實體本身)——這個「**%key** 開頭表識別、後面接內容」的命名慣例，跟 **failed/reported** 的 **%key/%msg** 是同一套規律，只是換了個欄位名稱

EML EXECUTE：跟 CREATE/UPDATE 語法不同的第三種呼叫方式

```

MODIFY ENTITIES OF zi_rc01_task
  ENTITY Task
  EXECUTE markDone
    FROM VALUE #( ( %key-task_id = lv_id ) )
  RESULT DATA(ls_result)
  FAILED DATA(ls_failed_action)
  REPORTED DATA(ls_reported_action).

```

- **EXECUTE <action名稱> FROM VALUE #((%key-... = ...))**——rc02 已經教過 **CREATE/UPDATE** 用 **FIELDS (...) WITH VALUE #(...)**、**DELETE** 用 **FROM VALUE #(...)**；**EXECUTE** 跟 **DELETE** 一樣用 **FROM**，因為 Action (至少 **markDone** 這種不帶輸入參數的) 只需要指定「對哪些實例執行」，不需要「宣告要改哪些欄位」
- ⚠ **RESULT** 在 **EML** 呼叫端直接是表格，不用像 **FAILED/REPORTED** 那樣加 **-<alias> : ls_result[1]-%param-status** 可以直接存取，但 **ls_failed_action/ls_reported_action** 要 **ls_failed_action-task** 才能存取——同一句 **EML** 陳述式裡，三個回應參數 (**RESULT/FAILED/REPORTED**) 的型別規則並不統一，要逐一確認，不能假設三個都一樣 (這條規則舊課程 rap08 在 Composition 情境也踩過一次，這次在 Action 情境又驗證了一次，是同一條規律的不同體現)

ABAP Unit 執行結果

```
Unit Test Results for CLAS/I ZCL_RC02_TASK_TEST.main
Status: ALL TESTS PASSED
Total: 3 | Passed: 3 | Failed: 0

[PASS] ZCL_RC02_TASK_TEST
  [PASS] CREATE_INVALID_STATUS (0.110s)
  [PASS] CREATE_UPDATE_DELETE (0.090s)
  [PASS] EXECUTE_MARK_DONE (0.080s)
```

新增的 `EXECUTE_MARK_DONE` 測試：CREATE 一筆 `status='0'` 的資料 → COMMIT → EXECUTE markDone → COMMIT → 驗證 `RESULT` 表格裡 `%param-status = 'D'`、`%param-description` 正確 → 額外做一次獨立的 `READ ENTITIES`，確認 `status='D'` 真的持久化到資料庫，不是只有 `RESULT` 看起來對而已。

學習目標

- 能寫出 BDEF Action 宣告的完整語法元素：`[internal][static][repeatable] action (...)` `ActionName [InputParameter] [OutputParameter];`，並解釋每個修飾詞的用途（即使這一課的範例都省略）
- 能講出 `result [1] $self` 三個部分各自的意義：`result` 關鍵字、`[1]` 基數、`$self` 型別
- 能寫出 Action Handler Method 的完整語法：`METHODS <名稱> FOR MODIFY IMPORTING keys FOR ACTION <alias>~<action名稱> RESULT result.`，並逐一解釋每個組成部分
- 能講出這一課最核心的結論：Action 的語法在這個 Cloud 環境跟舊 On-Premise 課程 rap07 完全相同（不像 rc03 的 Determination / Validation），根本原因是 Action 掛在 `FOR MODIFY` 這個沒有新舊版本落差的 Handler 類別底下，不是自己獨立的類別
- 能寫出 EML `EXECUTE` 的語法，知道跟 `CREATE/UPDATE ( FIELDS...WITH VALUE )`、`DELETE ( FROM VALUE )` 的差異與相似之處
- 知道 `RESULT` 在 EML 呼叫端直接是表格、不用加 `-<alias>`，但 `FAILED/REPORTED` 需要，三者型別規則不統一
- 能講出 Action 方法本體內部呼叫自己 Entity 一樣要 `IN LOCAL MODE`，這條規則不因為換了 RAP 機制（Determination/Validation → Action）而失效

物件清單

物件	名稱	型別	異動內容
Managed Behavior Definition	<code>ZI_RC01_TASK</code> (沿用 rc02/rc03)	BDEF/BDO	新增 <code>action markDone result [1] \$self;</code>
Implementation Class	<code>ZBP_I_RC01_TASK</code> (沿用 rc02/rc03)	CLAS/OC	Local Types <code>lhc_task</code> 新增 <code>markDone</code> 方法（使用者在 Eclipse 貼上）
ABAP Unit 測試類別	<code>ZCL_RC02_TASK_TEST</code> (沿用 rc02/rc03)	CLAS/OC	新增 <code>EXECUTE_MARK_DONE</code> 測試方法， <code>setup()</code> 一併補上 <code>RC02TEST03</code> 的清理

套件：ZRAPCLOUD。BDEF 的 Action 宣告與測試類別由 Claude 用 MCP 直接讀寫；Local Types 的 Handler Method (含 IMPORTING...FOR ACTION... 語法) 一樣由使用者在 Eclipse 貼上——這是這門課第三次遇到同一個 MCP 工具限制。

驗證方式

1. `get_abap_diagnostics` 確認 BDEF / ZBP_I_RC01_TASK / ZCL_RC02_TASK_TEST 三個物件都無語法錯誤 (BDEF 只宣告 Action、還沒補 Handler 時，`get_abap_diagnostics` 會回報明確的 WARNING: The action "ZI_RC01_TASK~MARKDONE" is not implemented.——注意是警告不是錯誤，跟 BDEF 本身可以獨立啟用一致)
2. `abap_activate` 全部回報 Activation successful
3. `run_unit_tests` 對 ZCL_RC02_TASK_TEST 執行，ALL TESTS PASSED (3/3，見上方完整輸出)

思考題

1. 這一課的 `markDone` 是「不帶輸入參數」的 Action。如果想讓它可以帶一個輸入參數 (例如指定完成時要附上的備註)，BDEF 跟 Handler Method 的簽章大概要怎麼改？ (提示：查官方文件 ABENBDL_ACTION_INPUT_PARAM，或對照這一課 OutputParameter / result 子句的語法模式，InputParameter 應該是類似的結構)
2. 這一課驗證了「Action 沒有新舊語法落差」是因為它掛在 FOR MODIFY 底下。如果之後要學 RAP 的 Function (官方文件裡跟 Action 平行、但用於「不改變任何實例狀態、純粹讀取或計算」的機制)，你會怎麼預測它有沒有新舊語法落差？該用什麼方法驗證？ (提示：回顧這一課「判斷方法」——先查這個語言元素屬於哪個 Handler 類別，再查那個類別的歷史)
3. `markDone` 的方法本體示範了「Action 呼叫 MODIFY ENTITIES 把合法值 'D' 寫入，不會被 validateStatus 擋下來」。如果反過來讓 `markDone` 內部故意寫入一個不合法的 status 值 (例如 'X')，你預期 EXECUTE `markDone` 的 COMMIT ENTITIES 會發生什麼事？跟 rc03 學到的「ROLLBACK ENTITIES」規則有沒有關係？

答案

見 `zi_rc01_task.bdef.abap` (含 `markDone` Action 宣告)、`zbp_i_rc01_task.clas.abap` (Local Types 新增 `markDone` 方法)、`zcl_rc02_task_test.clas.abap` (新增 `execute_mark_done` 測試方法)。SAP 端物件：ZI_RC01_TASK (BDEF)、ZBP_I_RC01_TASK (Implementation Class)、ZCL_RC02_TASK_TEST (ABAP Unit 測試類別)，套件 ZRAPCLOUD，`run_unit_tests` 執行結果：ALL TESTS PASSED (3/3)。